

Codex Master Agent Prompt

Ready-to-paste prompts and AGENTS.md rules

How to Use This PDF

Paste the master prompt into Codex at the start of the project. Then paste Phase 0. After each phase, test the project manually in Godot before continuing.

Master Codex Agent Prompt

You are the coding agent for a Godot 4 game project called Pacific Firestorm.

Project:

Pacific Firestorm is an original vertical-scrolling aerial shooter inspired by classic 1942-style arcade gameplay, but with a modern ultra-realistic Pacific air-combat visual direction.

Important:

This is not a direct clone of 1942. Do not copy Capcom branding, name, sprites, music, sound effects, level layouts, enemy waves, UI, or any copyrighted assets. Build an original spiritual successor.

Engine:

Godot 4.x

Language:

GDScript

Visual style:

Top-down or slight-tilt orthographic 3D. Use realistic aircraft, ocean, islands, smoke, fire, clouds, tracer rounds, explosions, and ships. Use placeholders first.

Gameplay style:

Arcade. Responsive. Readable. Not a realistic flight simulator.

Core rule:

Arcade gameplay. Realistic visuals. Original identity.

Technical approach:

Use a 3D world with an orthographic Camera3D. Gameplay occurs on the X/Z plane. Use simple Area3D hitboxes. Use data-driven JSON files for enemies, waves, stages, weapons, and power-ups. Use object pooling for bullets and effects once the basic loop works. Keep the project runnable after every phase.

Required systems:

- Main scene
- Player aircraft
- Player movement
- Player weapon system
- Player bullets
- Enemy base class
- Enemy movement patterns
- Enemy spawner
- Enemy bullets
- Bullet collision
- Player damage/lives
- Score system
- HUD
- Stage manager
- Game states
- Title screen
- Pause menu
- Game over screen
- Mission complete screen
- Power-ups
- Boss system
- Effects manager
- Audio manager
- Scrolling ocean/environment
- Export documentation

Development rules:

- Implement in phases.
- Do not over-engineer.
- Do not build all content before the core gameplay works.
- Do not silently remove existing functionality.
- Keep scripts small and readable.
- Document unfinished items in docs/known_issues.md.
- Update README and docs when behaviour changes.
- Prefer simple working code over clever architecture.
- Use placeholder assets until final assets are provided.
- Use fair arcade hitboxes, not exact model collision.
- Make bullets and hazards readable even with realistic graphics.

First task:

Create the repository structure, AGENTS.md, README.md, docs files, and basic Godot project skeleton. Do not implement full gameplay yet.

First Start Prompt

Start the Pacific Firestorm Godot project.

Use the full project rules below:

- Godot 4.x
- GDScript
- Original vertical scrolling aerial shooter
- Inspired by classic 1942-style arcade gameplay but not a direct clone
- Ultra-realistic top-down Pacific air-combat visual direction
- 3D orthographic gameplay presentation
- Arcade movement, not realistic flight simulation
- Data-driven stages/waves/enemies/weapons
- Placeholder assets first
- Keep project runnable after every phase

First task:

Create only the foundation:

1. AGENTS.md
2. README.md
3. docs/game_design.md
4. docs/art_direction.md
5. docs/technical_plan.md
6. docs/controls.md
7. docs/known_issues.md
8. docs/legal_notes.md
9. .github/pull_request_template.md
10. Godot folder structure
11. Minimal project.godot
12. scenes/main/Main.tscn
13. scripts/main/main.gd

Main.tscn should contain:

- root Node3D named Main
- WorldRoot Node3D
- Camera3D with orthographic projection
- DirectionalLight3D
- placeholder ocean Node3D
- PlayerSpawn Node3D
- EnemyRoot Node3D
- PlayerBulletRoot Node3D
- EnemyBulletRoot Node3D
- EffectsRoot Node3D
- UI placeholder CanvasLayer

Do not implement full gameplay yet.

Do not add copyrighted assets.

Use placeholders only.

Keep everything simple and readable.

Acceptance criteria:

- Project opens in Godot.
- Main.tscn runs.
- No script parse errors.
- README explains how to open the project.
- AGENTS.md contains the long-term rules for future Codex tasks.

AGENTS.md

AGENTS.md - Pacific Firestorm

Project Summary

Pacific Firestorm is an original Godot 4 vertical-scrolling aerial shooter inspired by classic arcade air-combat shooters, but with a modern ultra-realistic visual direction.

The game should feel like a simple, fast, readable arcade shooter. It should not become a realistic flight simulator.

Core Design Rule

Arcade gameplay. Realistic visuals. Original identity.

Legal / Creative Boundaries

Do not copy:

- Capcom branding
- 1942 name
- original 1942 sprites
- original 1942 sound effects
- original 1942 music
- exact 1942 enemy waves
- exact 1942 level layouts
- exact 1942 UI
- recognizable direct clones of copyrighted assets

Create original aircraft designs, enemy waves, UI, effects, levels, names, music, and sounds.

Engine

Use Godot 4.x.

Language

Use GDScript unless specifically instructed otherwise.

Game Type

Top-down / slight-tilt orthographic 3D vertical shooter.

Gameplay is 2D arcade logic on the X/Z plane. Visuals use 3D models, lighting, particles, and shaders.

Camera

Use Camera3D with orthographic projection. The camera looks downward at the battlefield. The gameplay plane is X/Z. Y is used for height/visual layering if needed.

Code Style

- Keep scripts small and readable.
- Prefer composition over giant manager scripts.
- Keep gameplay logic separated from rendering/effects.
- Use signals for major events where appropriate.
- Use data files for stages, enemies, waves, weapons, and power-ups.
- Keep the project runnable after every phase.
- Do not add advanced features before the core loop works.
- Do not over-engineer.

Required Core Systems

- Main scene
- Player aircraft
- Player movement
- Player shooting
- Player damage/lives
- Enemy base class
- Enemy spawner
- Enemy movement patterns
- Bullet system
- Collision detection
- Score system
- HUD
- Game states
- JSON wave loading
- Stage manager
- Effects manager
- Audio manager
- Menus

- Export documentation

Gameplay Feel

The game should be responsive and arcade-like.

Do not implement realistic:

- lift
- drag
- stalls
- fuel
- limited ammunition
- complex aerodynamics
- long turning radius

The player should move instantly and precisely.

Hitboxes

Use fair arcade-style hitboxes. The visual model may be large, but the actual player damage hitbox should be small and readable.

Readability

Even with realistic graphics, bullets, enemy fire, power-ups, and hazards must remain visible.

Use:

- tracer glow
- clear silhouettes
- UI highlights
- readable contrast
- consistent bullet colours

Performance Rules

Use object pooling for:

- player bullets
- enemy bullets
- explosions
- smoke trails
- hit sparks

Avoid:

- excessive real-time lights
- excessive transparent particles
- huge uncompressed textures
- high-poly models for small screen objects

Data-Driven Design

Use JSON for:

- enemy types
- weapon types
- power-up types
- stages
- enemy waves

Do not hard-code whole stages in scripts.

Development Discipline

For every task:

1. Make the smallest useful change.
2. Keep the project runnable.
3. Update documentation if behaviour changes.
4. Add clear notes to docs/known_issues.md if something is unfinished.
5. Do not silently remove existing functionality.

Acceptance Criteria for Every Phase

A phase is complete only when:

- the project opens in Godot
- there are no script parse errors
- the main scene runs
- the implemented feature works in a basic way
- obvious errors are documented or fixed